JOBSHEET 8
LOOPS 2

1. Objectives

- Understand the concept of nested loops in algorithms
- Understand the flow of nested loops
- Recognize the syntax of nested loops
- Be able to apply nested loops to solve a simple case study.

2. Practicum

2.1 Experiment 1: Review of Loops 1

Experiment time: 30 minutes

This experiment revisits the FOR loop before learning nested loops. The program prints the character `*` *n* times horizontally (side by side). Use *n* as a positive integer input.

Experiment Steps

1. Create a new repository named **daspro-jobsheet8** on GitHub, *clone* it to your computer, then open the folder in Visual Studio Code.
2. Create a new file **StarStudentNo.java** in that folder. Replace StudentNo with your student number, for example **Star05.java**.
3. Add the import statement for the class `Scanner` at the very top of the program.

```
import java.util.Scanner;
```

4. Create the class and the method `main()`.

```
public class StarStudentNo {
    public static void main(String[] args) {

    }
}
```

5. Inside the method `main()`, declare a `Scanner` object named `sc`, then display the instruction for entering the value to be stored in the variable `n`.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter the value of n = ");
int n = sc.nextInt();
```

6. On the next line, create a FOR loop to print `*` *n* times. Note that you use `print`, **not** `println`, because the character `*` must appear side by side without moving to a new line.

```
for (int i = 1; i <= n; i++) {
    System.out.print("*");
}
```

7. After the loop finishes, add `System.out.println();` so that the cursor moves to a new line, then close the Scanner object with `sc.close();`. The complete program looks as follows.

```
import java.util.Scanner;

public class StarStudentNo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the value of n = ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("*");
        }
        System.out.println();
        sc.close();
    }
}
```

8. Save the file, then *compile* and run the program.
9. Enter n = 5. Observe the result; the output should look like the display below (**bold blue text** is the input typed on the keyboard).

```
Enter the value of n = 5
*****
```

10. Run the program again with n = 3 and n = 7. Record the number of `*` characters printed in each test.
11. Once the program works as expected, *commit* and *push* the file **StarStudentNo.java** to the repository **daspro-jobsheet8**.

Questions

1. If the initialization `i = 1` is changed to `i = 0` without changing the condition, how many `*` characters are printed when n = 5? Explain.
2. If the condition `i <= n` is changed to `i < n`, how many times does the loop run when n = 5? Why is the result different?
3. Predict the output of the following code fragment when n = 9, then verify it by replacing the FOR loop in your program and running it.

```
for (int i = 2; i <= n; i += 2) {
    System.out.print(i + " ");
}
```

   Modify this code so that it prints the **odd** positive numbers that do not exceed n.
4. Explain why changing the increment `i++` to `i--` (with the condition still `i <= n`) makes the loop run endlessly, so that it does not stop in a reasonable time for positive n. Do not run that version; if it has already started running, stop the program by pressing **Ctrl + C** in the terminal.

2.2 Experiment 2: Rectangle Star Pattern (Nested FOR)

Experiment time: 30 minutes

In this experiment a *nested loop* of FOR statements is used to print a rectangle of stars. The number of rows and columns is entered separately, so the role of each loop can be observed clearly: the *outer loop* controls the **rows**, while the *inner loop* controls the **columns** (recall the shelf/locker analogy from the lecture material).

For the input rows = 3 and columns = 6, the output is:

```
******
******
******
```

Experiment Steps

1. Create a new file **RectangleStudentNo.java**.
2. Add `import java.util.Scanner;`, then create the class `RectangleStudentNo` together with the `main()` method, as in Experiment 1.
3. Inside `main()`, declare the `Scanner` named `sc`, then read two positive integers, namely `rows` and `columns`.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter the number of rows: ");
int rows = sc.nextInt();
System.out.print("Enter the number of columns: ");
int columns = sc.nextInt();
```

4. First write the loop that prints `*` `columns` times (as in Experiment 1). This loop will later become the inner loop.

```
for (int j = 1; j <= columns; j++) {
    System.out.print("*");
}
```

5. Run the program with the input rows = 3 and columns = 6. Notice that only **one line** of stars is printed, because there is no loop for the rows yet.

```
Enter the number of rows: 3
Enter the number of columns: 6
******
```

6. Wrap that loop in an outer loop that repeats `rows` times. Add `System.out.println();` **after** the inner loop (still inside the outer loop) so that each row of stars is printed on a new line.

```
for (int i = 1; i <= rows; i++) {       // outer loop: rows
    for (int j = 1; j <= columns; j++) {   // inner loop: columns
        System.out.print("*");
    }
    System.out.println();                  // new line
}
```

7. Close the Scanner with `sc.close();` at the end of the `main()` method. The complete program looks as follows.

```
import java.util.Scanner;

public class RectangleStudentNo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of rows: ");
        int rows = sc.nextInt();
        System.out.print("Enter the number of columns: ");
        int columns = sc.nextInt();

        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= columns; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

8. Save, *compile*, and run the program with the input rows = 3 and columns = 6. The output should look like the display below.

```
Enter the number of rows: 3
Enter the number of columns: 6
******
******
******
```

9. Run the program again for the inputs (rows, columns) = (4, 2) and (1, 5). Compare the shapes of the three outputs.

Questions

1. Explain the function of the variables `i` and `j` in the program.
2. How many times are the outer loop, the inner loop, and the statement `print("*")` executed for rows = 4 and columns = 7?
3. What happens if the bounds of the outer and inner loops are swapped (the outer loop uses `columns` and the inner loop uses `rows`)? Use rows = 3 and columns = 6 to explain.
4. Explain the effect of removing `System.out.println();` after the *inner loop*.
5. Modify the program so that the printed character is `#`, then *commit* and *push* the changes to GitHub.

2.3 Experiment 3: Debugging a Star Triangle – Nested WHILE

Experiment time: 45 minutes

The starting code in this experiment is intentionally written so that it does not yet produce the correct triangle format. Your task is to identify the errors and fix them without changing the WHILE structure into FOR.

Target output for n = 5:

```
*
**
***
****
*****
```

Experiment Steps

1. Create a new file **TriangleStudentNo.java**.
2. Type the following starter code completely (adjust the class name to match your file name).

```
import java.util.Scanner;

public class TriangleStudentNo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the value of n = ");
        int n = sc.nextInt();

        int i = 0;
        while (i <= n) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            i++;
        }
        sc.close();
    }
}
```

3. Save and run the program with n = 3, then with n = 5. Record the actual output, then compare it with the target output.
4. To find the cause of the error, add a temporary debug command right after the line `while (i <= n) {`. This command displays the value of `i` each time the *outer loop* starts.

```
while (i <= n) {
    System.out.print("[i=" + i + "] ");   // temporary debug
    int j = 0;
```

5. Run the program again with n = 5, then observe the result:
   a. how many times does the *outer loop* run? Is this the same as the desired number of rows?
   b. how many `*` characters are printed when `i = 0`?
   c. why are all the characters displayed on a single line?
6. Fix the code one step at a time, and run the program after each fix:

a. add `System.out.println();` after the *inner loop* finishes, before `i++`. Run it and observe: is there still any line that does not contain `*`?
b. change the initialization and/or the condition of the *outer loop* so that the loop runs exactly n times without any empty lines. Run it again.

7. Once the output matches the target output, **delete** the *debug* line(s). Run the program once more to confirm the result, then *commit* and *push* it to your repository.
8. Modify the program so that, after the triangle above is printed, it also prints an inverted triangle. Write a second nested WHILE below the first nested WHILE (still inside `main()`), without changing the WHILE into FOR.

> **Note:** the variable `i` has already been declared, so in the second nested WHILE it is enough to write `i = n;` without `int`. Writing `int i = n;` will cause the error *variable i is already defined*.

9. Run the program with n = 5. The inverted triangle section should look like the display below.

```
*****
****
***
**
*
```

Questions

1. Identify the mismatches in the initial output, and at least two fixes needed to produce a nested triangle pattern.
2. Explain the roles of the variables `i` and `j`. Why must the value of `j` be reset to 0 in every iteration of the *outer loop*? What happens if `j` is not reset?
3. Explain the changes to the initialization, the condition, and the update of the *outer loop* that are needed to form the inverted triangle.

2.4 Experiment 4: Case Study of Group Project Scores

Experiment time: 75 minutes

In assessing a group project, each group receives scores from a number of graders. The program receives the number of groups and the number of graders (the same for every group), then calculates the average score of each group and determines the group with the highest average. This case is solved with *nested loops* without storing all the scores: a **WHILE** (outer) for the groups, a **FOR** (middle) for the graders, and a **DO-WHILE** (inner) to repeat invalid score input.

Experiment Steps

1. Create a new file **GroupScoreStudentNo.java** containing `import java.util.Scanner;`, the class, the method `main()`, and a `Scanner` declaration named `sc`.
2. Ask for the number of groups and the number of graders. Both must be greater than 0. Use IF-ELSE so that the whole grading process runs only in the `else` block.

```
System.out.print("Enter the number of groups : ");
int numGroups = sc.nextInt();
System.out.print("Enter the number of graders: ");
int numGraders = sc.nextInt();

if (numGroups <= 0 || numGraders <= 0) {
    System.out.println("Groups and graders must be > 0.");
} else {
    // steps 3 to 8 are written inside this else block
}
sc.close();
```

3. Inside the `else` block, prepare the variables whose values must **persist** while all groups are processed.

```
int groupNo = 1;                    // outer loop counter
double highestAverage = -1;         // -1: the first group is recorded
int bestGroup = 0;
```

4. Create the outer loop using WHILE to process the groups one by one. At the start of each group, display the group number and *reset* `totalScore` to 0. Do not forget `groupNo++` at the end of the loop body; without this line the loop will never stop.

```
while (groupNo <= numGroups) {
    System.out.println("Group " + groupNo);
    double totalScore = 0;

    // inner loop over graders (step 5)

    // compute the average and check the best group (steps 6-7)

    groupNo++;
}
```

5. Replace the comment *inner loop over graders* with a FOR loop that reads the score of each grader and adds it to `totalScore`. Valid scores lie in the range 0–100. Use DO-WHILE so that the same score is requested again as long as it is outside the range.

```
for (int j = 1; j <= numGraders; j++) {
    int score;
    do {
        System.out.print("  Score from Grader " + j + ": ");
        score = sc.nextInt();
        if (score < 0 || score > 100) {
            System.out.println("  Score must be 0-100. Retry!");
        }
    } while (score < 0 || score > 100);
    totalScore += score;
}
```

6. After the *inner loop* finishes, calculate the average using the type `double` and display the result.

```
double averageScore = totalScore / numGraders;
System.out.println("Group " + groupNo
        + ": average score = " + averageScore);
```

7. Compare `averageScore` with `highestAverage`. Complete the following IF block yourself. If the averages are equal, the group found earlier remains the choice (because the operator `>` is used, not `>=`).

```
if (averageScore > highestAverage) {
    // complete: update highestAverage and bestGroup
}
```

8. After the closing brace of the WHILE (still inside the `else` block), display the best group and the highest average.

```
System.out.println("==============================");
System.out.println("Best group          : " + bestGroup);
System.out.println("Highest average     : " + highestAverage);
```

9. Save, *compile*, and run the program with 3 groups and 2 graders. Also test the validation by deliberately entering the score 105. The output should look like the display below.

```
Enter the number of groups : 3
Enter the number of graders: 2
Group 1
  Score from Grader 1: 80
  Score from Grader 2: 90
Group 1: average score = 85.0
Group 2
  Score from Grader 1: 75
  Score from Grader 2: 105
  Score must be 0-100. Retry!
  Score from Grader 2: 85
Group 2: average score = 80.0
Group 3
  Score from Grader 1: 90
  Score from Grader 2: 96
Group 3: average score = 93.0
==============================
Best group          : 3
Highest average     : 93.0
```

10. Run the program again with other variations, for example 2 groups and 3 graders, and test invalid input (number of groups = 0) to make sure the error message is displayed.
11. *Commit* and *push* the program code to GitHub.

Questions

1. Why must `totalScore` be reset inside the outer loop, but `highestAverage` must not be reset for each group?

2. For 3 groups and 2 graders (without any invalid input), how many times is the body of the FOR loop executed in total?
3. What happens if `totalScore` is declared as `int` and the average is calculated using integer division? Give an example of values.
4. Why is `highestAverage` given the initial value -1 rather than 0? Give an example of input that produces a wrong result if the initial value were 0.
5. This program has three levels of loops. Name each type of loop and its role.

3. Assignment

Working time: 120 minutes

Complete the following assignments independently. Use nested loops and selection structures where needed, and use only ordinary variables (accumulator and counter variables) to store the results of calculations. Create a separate file for each assignment. *Commit* and *push* all the files to the repository **daspro-jobsheet8** on GitHub.

1. Assignment 1: Nested Number Pattern

Write a program that prints a number pattern from 1 up to the row number. Enter an integer n (n is at least 1). The first row prints 1, the second row prints 1 2, and so on up to row n. After the pattern is complete, display the total number of digits that have been printed. Save the file with the name: **Task1StudentNo.java**.

For n = 5:

```
Enter the value of n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total numbers printed: 15
```

2. Assignment 2: Multiplication Table

Write a program that displays a multiplication table of size n × n using nested FOR or WHILE loops. The value of n is at least 3. Each position displays the product of its row number and column number. Arrange the spacing so that the output is neat, then display the sum of all the numbers in the table. Save the program with the file name: **Task2StudentNo.java**.

For n = 4:

```
Enter the value of n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Total of all numbers: 100
```

Hint: to align the numbers to the right, use `System.out.printf("%4d", result);`, which prints a number with a width of 4 characters, or use the tab character `"\t"` as a separator.

3. Assignment 3: Sales Recap for Several Stores

A shopping center has several stores. Each store serves a number of customers on the same day, and each customer buys several items. The program asks for the number of stores, then the number of customers at each store, and the number of items bought by each customer. Calculate and display the total number of customers and the total number of items for each store, the total number of items across all stores, and the number of the store with the most items. If several stores share the highest value, display the store with the smallest number. Save the program with the file name: **Task3StudentNo.java**.

Requirements: the number of stores is at least 1. The number of customers and the number of items per customer may be 0, but must not be negative. Use nested loops. The data does not need to be stored one by one; it is sufficient to accumulate it into the total variables.

Example: Store 1 has 2 customers who bought 3 and 2 items; Store 2 has 3 customers who bought 1, 4, and 2 items. The expected summary is:

```
Store 1 - customers: 2, total items: 5
Store 2 - customers: 3, total items: 7
Total items across all stores: 12
Store with the most items: 2
```